UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

Department of Cyber Security Engineering

Lab Report

Lab report no : 01

Name of the report : Windows Log Analysis with PowerShell

Course title : Threat Modeling and Security Monitoring Sessional

Course code : SEC 204

Submitted by: **Name:** Sujoy Devnath **ID:** 2404020 **Level:** 2 **Term:** 1 Department of Cyber Security Engineering, UFTB	**Submitted to:** **Md Masud Rana** **Lecturer**, Department of Cyber Security Engineering, UFTB

Windows Log Analysis with PowerShell

Part 1: Understanding Windows Security Event IDs

1. Version check

```
PS C:\WINDOWS\system32> $psversiontable.psversion

Major  Minor  Build  Revision
-----  -----  -----  --------
5      1      26100  9711
```

Part 2: Basic PowerShell Event Log Queries

1. View Recent Security Events

```
PS C:\WINDOWS\system32> get-winevent -logname security -maxevents 10


   ProviderName: Microsoft-Windows-Security-Auditing

TimeCreated                     Id LevelDisplayName Message
-----------                     -- --------------- -------
11/3/2026 11:50:19 PM         4672 Information     Special privileges assigned to new logon....
11/3/2026 11:50:19 PM         4624 Information     An account was successfully logged on....
11/3/2026 11:41:48 PM         4799 Information     A security-enabled local group membership was enumerated....
11/3/2026 11:41:48 PM         4799 Information     A security-enabled local group membership was enumerated....
11/3/2026 11:41:44 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 11:41:44 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 11:41:44 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 11:41:44 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 11:41:44 PM         5379 Information     Credential Manager credentials were read....
11/3/2026 11:41:44 PM         5379 Information     Credential Manager credentials were read....
```

2. Filter by Event ID

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4624} -MaxEvents 12


   ProviderName: Microsoft-Windows-Security-Auditing

TimeCreated                     Id LevelDisplayName Message
-----------                     -- ---------------- -------
11/3/2026 11:50:19 PM         4624 Information      An account was successfully logged on....
11/3/2026 11:41:44 PM         4624 Information      An account was successfully logged on....
11/3/2026 11:41:44 PM         4624 Information      An account was successfully logged on....
11/3/2026 11:41:27 PM         4624 Information      An account was successfully logged on....
11/3/2026 11:41:25 PM         4624 Information      An account was successfully logged on....
11/3/2026 11:40:07 PM         4624 Information      An account was successfully logged on....
11/3/2026 11:31:19 PM         4624 Information      An account was successfully logged on....
11/3/2026 11:31:18 PM         4624 Information      An account was successfully logged on....
11/3/2026 11:29:52 PM         4624 Information      An account was successfully logged on....
11/3/2026 11:29:51 PM         4624 Information      An account was successfully logged on....
11/3/2026 11:25:06 PM         4624 Information      An account was successfully logged on....
11/3/2026 11:20:37 PM         4624 Information      An account was successfully logged on....
```

3: Filter by Time Range

```
PS C:\WINDOWS\system32> $StartTime = (Get-Date).AddDays(-1)
>> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     StartTime=$StartTime
>> } -MaxEvents 10


   ProviderName: Microsoft-Windows-Security-Auditing

TimeCreated                     Id LevelDisplayName Message
-----------                     -- ---------------- -------
11/3/2026 11:55:07 PM         4672 Information      Special privileges assigned to new logon....
11/3/2026 11:55:07 PM         4624 Information      An account was successfully logged on....
11/3/2026 11:50:19 PM         4672 Information      Special privileges assigned to new logon....
11/3/2026 11:50:19 PM         4624 Information      An account was successfully logged on....
11/3/2026 11:41:48 PM         4799 Information      A security-enabled local group membership was enumerated....
11/3/2026 11:41:48 PM         4799 Information      A security-enabled local group membership was enumerated....
11/3/2026 11:41:44 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 11:41:44 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 11:41:44 PM         5379 Information      Credential Manager credentials were read....
11/3/2026 11:41:44 PM         5379 Information      Credential Manager credentials were read....
```

4. Count Events

```
PS C:\WINDOWS\system32> $StartTime = (Get-Date).AddDays(-1)
>> $FailedLogons = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=$StartTime
>> }
>> $FailedLogons.Count
29
```

Part 3: Extracting Event Properties

1. Understanding Event Structure

```
PS C:\WINDOWS\system32> $Event = Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4625} -MaxEvents 1
>> $Event | Format-List *


Message             : An account failed to log on.

                      Subject:
                       Security ID:           S-1-5-21-1242217923-602775437-513446711-1001
                       Account Name:          SUJOY
                       Account Domain:        DESKTOP-GFT9DR7
                       Logon ID:              0x65127

                      Logon Type:                    2

                      Account For Which Logon Failed:
                       Security ID:           S-1-0-0
                       Account Name:          SUJOY
                       Account Domain:        DESKTOP-GFT9DR7

                      Failure Information:
                       Failure Reason:        Unknown user name or bad password.
                       Status:                0xC000006E
                       Sub Status:            0xC000006E
```

2. Extract Data from Failed Logons

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4625} -MaxEvents 10 |
>>     ForEach-Object {
>>         [PSCustomObject]@{
>>             TimeCreated = $_.TimeCreated
>>             TargetUser = $_.Properties[5].Value
>>             SourceIP = $_.Properties[19].Value
>>             LogonType = $_.Properties[8].Value
>>             FailureReason = $_.Properties[13].Value
>>         }
>>     } | Format-Table -AutoSize

TimeCreated           TargetUser SourceIP LogonType FailureReason
-----------           ---------- -------- --------- -------------
11/3/2026 11:14:31 PM SUJOY      -        %%2313    DESKTOP-GFT9DR7
11/3/2026 10:58:23 PM SUJOY      -        %%2313    DESKTOP-GFT9DR7
11/3/2026 4:34:14 PM  SUJOY      -        %%2313    DESKTOP-GFT9DR7
11/3/2026 4:34:12 PM  SUJOY      -        %%2313    DESKTOP-GFT9DR7
11/3/2026 3:03:22 PM  SUJOY      -        %%2313    DESKTOP-GFT9DR7
11/3/2026 2:46:45 PM  SUJOY      -        %%2313    DESKTOP-GFT9DR7
11/3/2026 2:46:40 PM  SUJOY      -        %%2313    DESKTOP-GFT9DR7
11/3/2026 2:43:18 PM  SUJOY      -        %%2313    DESKTOP-GFT9DR7
11/3/2026 2:42:07 PM  SUJOY      -        %%2313    DESKTOP-GFT9DR7
11/3/2026 1:43:49 PM  SUJOY      -        %%2313    DESKTOP-GFT9DR7
```

3. Analyze Failed Logon Attempts

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     $_.Properties[19].Value
>> } | Where-Object { $_ -ne '-' -and $_ -ne '' } |
>>     Group-Object |
>>     Sort-Object Count -Descending |
>>     Select-Object -First 5 Name, Count
PS C:\WINDOWS\system32>
```

4. Identify Targeted Accounts

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     $_.Properties[5].Value  # TargetUserName
>> } | Where-Object { $_ -ne '-' -and $_ -ne '' } |
>>     Group-Object |
>>     Sort-Object Count -Descending |
>>     Select-Object -First 5 @{Name='Username';Expression={$_.Name}}, @{Name='Failed Attempts';Expression={$_.Count}} |
>>     Format-Table -AutoSize

Username Failed Attempts
-------- ---------------
SUJOY                 29
```

5. Analyze Logon Types

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     $LogonType = $_.Properties[8].Value
>>     switch ($LogonType) {
>>         2 { 'Interactive' }
>>         3 { 'Network' }
>>         4 { 'Batch' }
>>         5 { 'Service' }
>>         7 { 'Unlock' }
>>         8 { 'NetworkCleartext' }
>>         9 { 'NewCredentials' }
>>         10 { 'RemoteInteractive' }
>>         11 { 'CachedInteractive' }
>>         default { "Unknown ($LogonType)" }
>>     }
>> } | Group-Object |
>>     Sort-Object Count -Descending |
>>     Select-Object @{Name='Logon Type';Expression={$_.Name}}, @{Name='Count';Expression={$_.Count}} |
>>     Format-Table -AutoSize

Logon Type       Count
----------       -----
Unknown (%%2313)    29
```

Part 4: Detecting Brute-Force Attacks

1. Identify Brute-Force Patterns

```
PS C:\WINDOWS\system32> # Define threshold
>> $FailureThreshold = 5
>> $TimeWindow = (Get-Date).AddHours(-1)
>>
>> # Get failed logons
>> $FailedLogons = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=$TimeWindow
>> } | ForEach-Object {
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         SourceIP = $_.Properties[19].Value
>>         TargetUser = $_.Properties[5].Value
>>         LogonType = $_.Properties[8].Value
>>     }
>> }
>>
>> # Group by source IP
>> $SuspiciousIPs = $FailedLogons |
>>     Where-Object { $_.SourceIP -ne '-' } |
>>     Group-Object SourceIP |
>>     Where-Object { $_.Count -ge $FailureThreshold } |
>>     Sort-Object Count -Descending
>>
>> # Display results
>> Write-Host "`n=== POTENTIAL BRUTE-FORCE ATTACKS DETECTED ===" -ForegroundColor Red
>> Write-Host "Time Window: Last 1 hour" -ForegroundColor Yellow
>> Write-Host "Threshold: $FailureThreshold failed attempts`n" -ForegroundColor Yellow
>>
>> foreach ($IP in $SuspiciousIPs) {
>>     Write-Host "Source IP: $($IP.Name)" -ForegroundColor Red
>>     Write-Host "Failed Attempts: $($IP.Count)" -ForegroundColor Red
>>     Write-Host "Targeted Accounts:" -ForegroundColor Yellow
>>     $IP.Group | Select-Object -ExpandProperty TargetUser -Unique | ForEach-Object {
>>         Write-Host "  - $_"
>>     }
>>     Write-Host ""
>> }
Get-WinEvent : No events were found that match the specified selection criteria.
At line:6 char:17
+ $FailedLogons = Get-WinEvent -FilterHashtable @{
+                 ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand


=== POTENTIAL BRUTE-FORCE ATTACKS DETECTED ===
Time Window: Last 1 hour
Threshold: 5 failed attempts
```

2. Generate Test Failed Logons

```
PS C:\WINDOWS\system32> # This will generate Event ID 4625
>> 1..10 | ForEach-Object {
>>     # Attempt to access a network share with wrong credentials
>>     net use \\localhost\C$ /user:FakeUser WrongPassword 2>$null
>>     Start-Sleep -Seconds 2
>> }
PS C:\WINDOWS\system32> ^C
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddMinutes(-5)
>> } | Select-Object TimeCreated, Message | Format-List


TimeCreated : 11/4/2026 12:40:00 AM
Message     : An account failed to log on.

              Subject:
                Security ID:            S-1-0-0
                Account Name:           -
                Account Domain:         -
                Logon ID:               0x0

              Logon Type:                     3

              Account For Which Logon Failed:
                Security ID:            S-1-0-0
                Account Name:           FakeUser
                Account Domain:         -
```

Part 5: Detecting Lateral Movement

1. Detect Explicit Credential Usage

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4648
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     $xml = [xml]$_.ToXml()
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         SourceUser = $xml.Event.EventData.Data[1].'#text'
>>         TargetUser = $xml.Event.EventData.Data[5].'#text'
>>         TargetServer = $xml.Event.EventData.Data[8].'#text'
>>         Process = $xml.Event.EventData.Data[11].'#text'
>>     }
>> } | Format-Table -AutoSize

TimeCreated           SourceUser       TargetUser TargetServer    Process
-----------           ----------       ---------- ------------    -------
11/4/2026 12:40:00 AM SUJOY            FakeUser   DESKTOP-GFT9DR7 -
11/4/2026 12:39:27 AM SUJOY            FakeUser   DESKTOP-GFT9DR7 -
11/3/2026 10:55:02 PM DESKTOP-GFT9DR7$ SUJOY      localhost       C:\Windows\System32\svchost.exe
11/3/2026 10:55:00 PM DESKTOP-GFT9DR7$ DWM-1      localhost       C:\Windows\System32\winlogon.exe
11/3/2026 10:55:00 PM DESKTOP-GFT9DR7$ UMFD-1     localhost       C:\Windows\System32\winlogon.exe
11/3/2026 10:55:00 PM DESKTOP-GFT9DR7$ UMFD-0     localhost       C:\Windows\System32\wininit.exe
11/3/2026 4:33:37 PM  DESKTOP-GFT9DR7$ SUJOY      localhost       C:\Windows\System32\svchost.exe
11/3/2026 4:33:35 PM  DESKTOP-GFT9DR7$ DWM-1      localhost       C:\Windows\System32\winlogon.exe
```

2. Detect Network Share Access

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=5140
>>     StartTime=(Get-Date).AddHours(-1)
>> } | ForEach-Object {
>>     $xml = [xml]$_.ToXml()
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         Account = $xml.Event.EventData.Data[1].'#text'
>>         SourceIP = $xml.Event.EventData.Data[3].'#text'
>>         ShareName = $xml.Event.EventData.Data[4].'#text'
>>     }
>> } | Format-Table -AutoSize
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:1
+ Get-WinEvent -FilterHashtable @{
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand
```

Part 6: Detecting Privilege Escalation

1. Monitor Admin Group Changes

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4732
>>     StartTime=(Get-Date).AddDays(-7)
>> } | ForEach-Object {
>>     $xml = [xml]$_.ToXml()
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         AddedUser = $xml.Event.EventData.Data[0].'#text'
>>         GroupName = $xml.Event.EventData.Data[2].'#text'
>>         AddedBy = $xml.Event.EventData.Data[6].'#text'
>>     }
>> } | Format-Table -AutoSize
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:1
+ Get-WinEvent -FilterHashtable @{
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand
```

2. Detect Special Privilege Assignments

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4672
>>     StartTime=(Get-Date).AddHours(-1)
>> } | ForEach-Object {
>>     $xml = [xml]$_.ToXml()
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         Account = $xml.Event.EventData.Data[1].'#text'
>>         LogonID = $xml.Event.EventData.Data[3].'#text'
>>     }
>> } | Where-Object { $_.Account -notlike '*$' } |  # Filter out system accounts
>>     Format-Table -AutoSize

TimeCreated           Account LogonID
-----------           ------- -------
11/4/2026 1:11:19 AM  SYSTEM  0x3e7
11/4/2026 1:11:17 AM  SYSTEM  0x3e7
11/4/2026 1:10:07 AM  SYSTEM  0x3e7
11/4/2026 1:00:05 AM  SYSTEM  0x3e7
11/4/2026 1:00:03 AM  SYSTEM  0x3e7
11/4/2026 12:55:07 AM SYSTEM  0x3e7
11/4/2026 12:55:04 AM SYSTEM  0x3e7
11/4/2026 12:52:41 AM SYSTEM  0x3e7
11/4/2026 12:40:07 AM SYSTEM  0x3e7
11/4/2026 12:31:28 AM SYSTEM  0x3e7
11/4/2026 12:31:27 AM SYSTEM  0x3e7
11/4/2026 12:25:07 AM SYSTEM  0x3e7
```

Part 7: Creating Comprehensive Analysis Scripts

1. Create a Complete Security Analysis Script

Analyze-SecurityLogs.ps1

```powershell
<#
.SYNOPSIS
    Comprehensive Windows Security Log Analysis Script
.DESCRIPTION
    Analyzes Windows Security Event Logs for suspicious activity
.PARAMETER Hours
    Number of hours to analyze (default: 24)
.EXAMPLE
    .\Analyze-SecurityLogs.ps1 -Hours 24
#>

param(
    [int]$Hours = 24
)

$StartTime = (Get-Date).AddHours(-$Hours)

Write-Host "`n========================================" -ForegroundColor Cyan
Write-Host "Windows Security Log Analysis Report" -ForegroundColor Cyan
Write-Host "========================================" -ForegroundColor Cyan
Write-Host "Analysis Period: Last $Hours hours" -ForegroundColor Yellow
Write-Host "Start Time: $StartTime" -ForegroundColor Yellow
```

```
PS E:\UFTB> .\Analyze-SecurityLogs.ps1 -Hours 24

========================================
Windows Security Log Analysis Report
========================================
Analysis Period: Last 24 hours
Start Time: 11/03/2026 01:26:02
========================================

[1] FAILED LOGON ANALYSIS
------------------------------------
Total Failed Logons: 34

Top 5 Source IPs:

IP Address Attempts
---------- --------
::1                2
```

Part 8: Exporting Results

1. Export to CSV

```
PS C:\> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         SourceIP = $_.Properties[19].Value
>>         TargetUser = $_.Properties[5].Value
>>         LogonType = $_.Properties[8].Value
>>         FailureReason = $_.Properties[13].Value
>>     }
>> } | Export-Csv -Path "FailedLogons.csv" -NoTypeInformation
>>
>> Write-Host "Exported to FailedLogons.csv"
Exported to FailedLogons.csv
```

2. Export to HTML Report

```
PS C:\> $FailedLogons = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         SourceIP = $_.Properties[19].Value
>>         TargetUser = $_.Properties[5].Value
>>         LogonType = $_.Properties[8].Value
>>     }
>> }
>>
>> $HTML = $FailedLogons | ConvertTo-Html -Title "Failed Logon Report" -PreContent "<h1>Failed Logon Analysis</h1><p>Generated
: $(Get-Date)</p>"
>>
>> $HTML | Out-File "FailedLogons.html"
>>
>> Write-Host "Exported to FailedLogons.html"
Exported to FailedLogons.html
```